Halo 系统完整代码技术文档

代码分析文档

2026 年 1 月 25 日

目录

I 系统概述 **5**

1 文档概述 **5**

2 系统架构 **5**

3 系统工作流程 **5**

3.1 整体流程 5

3.2 数据流 6

II Components 模块 **6**

4 Components 模块概述 **6**

5 Operator 类 **6**

5.1 类定义位置 6

5.2 功能概述 7

5.3 关键属性 7

5.4 详细说明 7

5.4.1 __init__ 方法 7

5.4.2 运行时字段 7

5.5 Benchmark 类 8

6 Query 类 **8**

6.1 类定义位置 8

6.2 功能概述 8

6.3 关键属性 8

6.4 op_output 字典 9

7 ModelConfig 类 **9**
7.1 类定义位置 9
7.2 功能概述 9
7.3 关键属性 10

8 ExecuteInfo 类 **10**
8.1 类定义位置 10
8.2 功能概述 10
8.3 类定义 10

III Parser 模块 **10**

9 Parser 模块概述 **11**

10 YAML 配置格式 **11**
10.1 基本结构 11

11 核心函数 **12**
11.1 build_ops_from_config 12
11.1.1 执行流程 12
11.2 _compute_max_distances 12
11.2.1 功能 12
11.2.2 算法：DFS + 记忆化 12

IV Schedulers 模块 **12**

12 Schedulers 模块概述 **14**

13 调度器接口 **14**

14 调度策略详解 **14**
14.1 schedule_rr：轮询调度 14
14.2 schedule_by_levels：层级调度 15
14.3 schedule_colocate_parents：父节点共置调度 16
14.4 schedule_dp：数据并行调度 16
14.5 schedule_heuristic：启发式调度 16
14.6 schedule_search：Beam-Search 调度 16

15 调度器对比 16

V Optimizers 模块 16

16 Optimizers 模块概述 17

17 Optimizer 类 17
17.1 初始化 17
17.2 execute 方法 17
17.2.1 依赖关系保证机制 18
17.2.2 关键辅助函数 18

VI Workers 模块 18

18 Workers 模块概述 18

19 vLLMWorker 类 18
19.1 初始化 18
19.2 init_op 方法 19
19.2.1 模型切换逻辑 19
19.3 execute 方法 19
19.3.1 执行流程 19
19.4 run 方法 19

VII Util 模块 19

20 Util 模块概述 20

21 函数详解 20
21.1 _resolve_dtype 20
21.2 _visible_physical_gpu_ids 20

VIII 使用指南 20

22 基本使用 20

23 性能优化建议 21
23.1 调度策略选择 21

24 总结 **21**

Part I

系统概述

1 文档概述

本文档是 Halo 系统的完整技术文档，详细分析了系统的各个模块，从函数级别到语句级别解释了代码的作用和实现逻辑。

2 系统架构

Halo 系统采用分层架构：

1. **配置层**：YAML 文件定义工作流
2. **解析层**：Parser 将配置转换为 DAG
3. **调度层**：Scheduler 生成执行计划
4. **协调层**：Optimizer 管理多进程执行
5. **执行层**：Worker 执行实际推理

3 系统工作流程

3.1 整体流程

1. **配置加载**：从 YAML 文件加载工作流定义
2. **图构建**：Parser 解析配置，构建 Operator 对象图
3. **调度规划**：根据选定的调度策略，生成执行计划
4. **执行协调**：Optimizer 创建多个 Worker 进程，按计划派发任务
5. **模型推理**：Worker 接收任务，执行批量推理
6. **结果收集**：Optimizer 收集所有 Worker 的结果，更新 Query 状态

3.2 数据流

```
YAML 配置
    ↓
Parser → Operator 图
    ↓
Scheduler → Workflows (执行计划)
    ↓
Optimizer → ExecuteInfo (任务封装)
    ↓
Worker → 推理结果
    ↓
Optimizer → 更新 Query.op\_output
    ↓
最终结果
```

Part II

Components 模块

4 Components 模块概述

components 模块定义了 Halo 系统的核心数据结构，包括：

- Operator：表示工作流中的一个操作节点
- Query：表示一个用户查询请求
- ModelConfig：模型配置信息
- ExecuteInfo：执行信息封装

5 Operator 类

5.1 类定义位置

halo/components/operator.py

5.2 功能概述

Operator 类表示智能体工作流图中的一个节点（OP），每个节点对应一个模型推理任务，包含模型配置、输入输出依赖关系等信息。

5.3 关键属性

```
class Operator:
    def __init__(self, id=None, prompt=None, model_config=None,
        keep_cache=False):
        self.id = id if id is not None else uuid.uuid4()
        self.input_ops = []                # 前驱节点列表
        self.output_nodes = []             # 后继节点列表
        self.prompt = prompt               # 提示词（可选）
        self.model_config = model_config # 模型配置对象
        self.max_distance = None           # 到终点的最长距离
        self.keep_cache = keep_cache       # 是否保留 KV cache
```

5.4 详细说明

5.4.1 ___init___ 方法

功能 初始化一个 Operator 对象

参数

- `id`：操作符的唯一标识符，如果为 None 则自动生成 UUID
- `prompt`：可选的提示词字符串
- `model_config`：`ModelConfig` 对象，包含模型相关配置
- `keep_cache`：布尔值，指示是否保留 KV cache 供下游节点使用

5.4.2 运行时字段

- `data_parallel`：标记该 OP 是否采用数据并行（多设备复制执行）
- `is_duplicate`：标记是否为复制节点（用于数据并行）
- `duplicate_info`：复制信息 `[rep_idx, rep_total]`
- `main_node`：如果为复制节点，指向原始主节点
- `benchmark`：性能基准对象，记录 `init_time`、`prefill_time`、`generate_time`
- `max_distance`：由 parser 计算，表示到任意终点 OP 的最长路径长度

5.5 Benchmark 类

Operator 内部包含一个 Benchmark 类，用于记录性能指标：

```
class Benchmark:
    def __init__(self):
        self.init_time = 0        # 初始化时间
        self.prefill_time = 0     # Prefill 阶段时间
        self.generate_time = 0    # Generate 阶段时间

    def total_time(self):
        return self.init_time + self.prefill_time + self.generate_time

    def update(self, dict):
        self.init_time += dict.get('init_time', 0.0)
        self.prefill_time += dict.get('prefill_time', 0.0)
        self.generate_time += dict.get('generate_time', 0.0)
```

6 Query 类

6.1 类定义位置

halo/components/query.py

6.2 功能概述

Query 类表示一个用户查询请求，包含查询 ID、提示词、执行状态、中间结果等信息。

6.3 关键属性

```
class Query:
    def __init__(self, id, prompt, priority=0):
        self.id = id                        # 查询唯一标识
        self.prompt = prompt                # 用户输入的提示词
        self.prompt_len = len(prompt)       # 提示词长度
        self.status = "pending"             # 状态: pending/running/finished
        self.priority = priority            # 优先级（数值越大优先级越高）
```

```python

        # 生成相关
        self.prefix_len = 0              # 前缀长度（用于 cache）
        self.input_ids = None            # Tokenized 输入 ID
        self.generated_tokens = 0         # 已生成 token 数
        self.decoded_tokens = ""         # 已解码的文本
        self.new_tokens = 0              # 本轮新生成的 token 数

        # 工作流执行相关
        self.op_output = {}               # 字典: op_id -> 该 OP 的输出文本
        self.step = 0                     # 当前执行步骤
        self.cache = None                 # KV cache（用于 TransformersWorker）

        # 性能基准
        self.create_time = time.perf_counter()  # 创建时间戳
        self.benchmark = {}               # 字典: op_id -> (start_time, end_time)
```

6.4 op_output 字典

这是 Query 类的核心字段之一，用于实现工作流中的依赖关系：

- **键**：OP 的 ID（字符串）
- **值**：该 OP 对该查询产生的输出文本
- **用途**：当执行一个 OP 时，系统会检查其所有父 OP 的输出是否都在 op_output 中，如果存在，则将这些输出拼接到当前 OP 的 prompt 后面，实现多步推理

7 ModelConfig 类

7.1 类定义位置

halo/components/model_config.py

7.2 功能概述

ModelConfig 类封装了模型推理所需的所有配置参数。

7.3 关键属性

```
class ModelConfig:
    def __init__(
        self,
        model_name,                # 模型名称
        system_prompt=None,         # 系统提示词
        temperature=0.7,           # 采样温度
        top_p=0.9,                 # Top-p 采样参数
        max_tokens=256,            # 最大生成 token 数
        max_batch_size=8,          # 最大批处理大小
        dtype='bfloat16',          # 数据类型
        use_chat_template=True,    # 是否使用聊天模板
        quantization=None,         # 量化配置
        lora_config=None,          # LoRA 配置
        max_model_len=None,         # 最大模型长度
        min_tokens=0,              # 最小生成 token 数
    ):
        # 所有参数直接赋值给实例属性
```

8 ExecuteInfo 类

8.1 类定义位置

halo/components/exe_info.py

8.2 功能概述

ExecuteInfo 类是一个简单的数据容器，用于封装一次执行任务所需的所有信息。

8.3 类定义

```
class ExecuteInfo:
    def __init__(self, op: Operator, query_ids, prompts):
        self.op = op                        # 要执行的 Operator
        self.query_ids = query_ids          # 查询 ID 列表
        self.prompts = prompts              # 对应的提示词列表
```

Part III

Parser 模块

9 Parser 模块概述

parser.py 模块负责从 YAML 配置文件解析并构建智能体工作流的有向无环图（DAG）。

10 YAML 配置格式

10.1 基本结构

```yaml
start_ops:
  - op0          # 起始节点列表
end_ops:
  - op2          # 终止节点列表

ops:
  op0:
    model: meta-llama/Llama-3.1-8B-Instruct
    prompt: "Please answer the question."
    max_tokens: 512
    input_ops: []           # 空列表表示起始节点
    output_ops:
      - op1

  op1:
    model: meta-llama/Llama-3.1-8B-Instruct
    prompt: "Please refine the answer."
    max_tokens: 1024
    input_ops:
      - op0      # 依赖 op0 的输出
    output_ops:
      - op2
```

11 核心函数

11.1 build_ops_from_config

这是 parser 模块的核心函数，负责将配置字典转换为可执行的 `Operator` 对象图。

11.1.1 执行流程

1. **验证必需字段**：检查 `ops`、`start_ops`、`end_ops` 是否存在且非空
2. **创建 Operator 对象**：为每个 OP ID 创建一个空的 `Operator` 对象
3. **验证引用和必需字段**：检查每个 OP 是否有 `model` 字段，引用的 OP ID 是否存在
4. **建立依赖关系并配置模型**：
   - 建立双向链接：`op.input_ops` 和 `op.output_ops`
   - 自动推断 `keep_cache`：如果后继节点使用相同模型，则自动设为 True
   - 创建 `ModelConfig` 对象
5. **初始化运行时字段**：设置 `data_parallel`、`is_duplicate` 等
6. **解析 start_ops 和 end_ops**：转换为 `Operator` 对象列表
7. **计算最长距离**：调用 `_compute_max_distances` 为每个 OP 计算到终点的最长路径

11.2 _compute_max_distances

11.2.1 功能

为每个 OP 计算到任意终点 OP 的最长路径长度（边数），同时检测循环依赖。

11.2.2 算法：DFS + 记忆化

使用深度优先搜索（DFS）和记忆化技术，时间复杂度 $O(V + E)$。

Algorithm 1 计算最长距离到终点

```
procedure DFS(op, end_set, memo, visiting)
    if op in memo then
        return memo[op]
    end if
    if op in visiting then
        raise "Cycle detected"
    end if
    visiting.add(op)
    if op in end_set then
        memo[op] = 0
        visiting.remove(op)
        return 0
    end if
    best = -1
    for child in op.output_ops do
        d = DFS(child, end_set, memo, visiting)
        if d != -1 then
            best = max(best, d + 1)
        end if
    end for
    memo[op] = best
    visiting.remove(op)
    return best
end procedure
```

Part IV

Schedulers 模块

12 Schedulers 模块概述

schedulers 模块实现了多种调度策略，用于将工作流图中的 Operator 分配到多个 GPU 设备上执行。

13 调度器接口

所有调度器都遵循相同的接口：

```
def schedule_xxx(
    device_cnt: int,            # 设备数量
    start_ops: List[Operator],  # 起始节点列表
    end_ops: List[Operator],    # 终止节点列表
    all_ops: List[Operator],    # 所有 OP 列表
    queries: List[Query],       # 查询列表
    **kwargs                    # 其他参数
) -> List[List[Dict]]:
    """
    返回 workflows，格式为：
    List[ per-device List[ {"command": "execute", "params": (op,
        query_ids)} ] ]
    """
```

14 调度策略详解

14.1 schedule_rr：轮询调度

最简单的调度策略：按照拓扑顺序，将每个 OP 轮询分配到设备上。

```
def schedule_rr(
    device_cnt: int,
    all_ops: List[Operator],
    queries: List[Query],
) -> List[List[Dict]]:
    """Round-Robin per op; each op handles ALL queries."""
    workflows: List[List[Dict]] = [[] for _ in range(device_cnt)]
```

```
    all_ids = [q.id for q in queries]
    topo = _topo_order(all_ops)

    d = 0
    for op in topo:
        workflows[d].append({"command": "execute", "params": (op, list(all_ids))})
        d = (d + 1) % device_cnt
    return workflows
```

14.2 schedule_by_levels：层级调度

将 OP 按依赖层级分组，同一层级的 OP 可以并行执行。

```
def schedule_by_levels(
    device_cnt: int,
    all_ops: List[Operator],
    queries: List[Query],
) -> List[List[Dict]]:
    """Level-based RR: assign ops within the same topo level round-robin."""
    workflows: List[List[Dict]] = [[] for _ in range(device_cnt)]
    all_ids = [q.id for q in queries]
    topo = _topo_order(all_ops)

    # 计算每个 OP 的层级
    level = {}
    for op in topo:
        level[op] = 0 if not op.input_ops else max(level[p] for p in op.input_ops) + 1

    # 按层级分配
    d = 0
    for lv in sorted(set(level.values())):
        layer_ops = [op for op in topo if level[op] == lv]
        for op in layer_ops:
            workflows[d % device_cnt].append({
                "command": "execute",
                "params": (op, list(all_ids))
            })
```

```
            d += 1
    return workflows
```

14.3 schedule_colocate_parents：父节点共置调度

优先将 OP 分配到其父节点所在的设备，以提高缓存局部性。

14.4 schedule_dp：数据并行调度

将查询 ID 固定分片到各设备，每个 OP 在所有设备上并行执行。

14.5 schedule_heuristic：启发式调度

结合多种优化技术的复杂调度策略：

- 依赖感知的前沿扩展
- 基于 `max_distance` 的优先级排序
- 数据并行复制（当设备数多于就绪 OP 数时）
- 父节点设备优先分配（缓存局部性）

14.6 schedule_search：Beam-Search 调度

使用 Beam-Search 算法搜索近似最优的设备分配方案，考虑模型切换成本、负载均衡等因素。

15 调度器对比

调度器	并行执行	模型切换优化	缓存局部性	复杂度
RR	否	否	否	低
By Levels	是（层级内）	否	否	低
Colocate Parents	否	否	是	中
Data Parallel	是	否	否	中
Heuristic	是	部分	是	高
Beam-Search	是	是	是	很高

表 1: 调度器特性对比

Part V

Optimizers 模块

16 Optimizers 模块概述

optimizers 模块是 Halo 系统的核心协调器，负责：

- 从 YAML 配置构建工作流图
- 创建和管理多个 Worker 进程（每个 GPU 一个）
- 调用调度器生成执行计划
- 协调 Worker 执行任务，保证依赖关系
- 收集结果并更新 Query 状态

17 Optimizer 类

17.1 初始化

```python
def __init__(self, config_path: str, **kwargs):
    self.config = load_config(config_path)
    mp.set_start_method("spawn", force=True)

    self.device_cnt = torch.cuda.device_count()
    self.processes: List[mp.Process] = []
    self.cmd_queues: List[mp.Queue] = []
    self.result_queues: List[mp.Queue] = []

    # 构建 OP 图
    self.ops, self.start_ops, self.end_ops, self.models =
        build_ops_from_config(self.config)
    self._create_workers()
```

17.2 execute 方法

这是 Optimizer 的核心方法，负责协调所有 Worker 执行任务。

17.2.1 依赖关系保证机制

- **依赖检查**：在派发任务前，检查该任务的所有父 OP 是否已完成
- **阻塞处理**：如果依赖未满足，任务不会派发，等待父任务完成
- **自动重试**：每当有任务完成，会重新检查所有等待中的任务
- **死锁避免**：通过定期重试和依赖检查，避免系统死锁

17.2.2 关键辅助函数

__cmd__transfer：将调度器生成的 task 转换为 Worker 可执行的格式，拼接父节点的输出。

__task__ready：检查任务是否满足依赖关系，所有父 OP 的输出必须已就绪。

__try__send：尝试向 Worker 发送下一个任务，只有在依赖满足时才发送。

Part VI

Workers 模块

18 Workers 模块概述

`workers` 模块实现了实际的模型推理执行层。Halo 支持两种 Worker 实现：

- `vLLMWorker`：基于 vLLM 的高性能推理引擎
- `TransformersWorker`：基于 Transformers 库的推理引擎

19 vLLMWorker 类

19.1 初始化

```python
def __init__(
    self,
    id: int,
    physical_gpu_id: int,
    cmd_queue: "queue.Queue",
    result_queue: "queue.Queue",
) -> None:
    self.id = id
```

```
    self.device = f"cuda:{physical_gpu_id}"

    # 绑定到特定 GPU
    os.environ["CUDA_VISIBLE_DEVICES"] = str(physical_gpu_id)

    # 运行时状态
    self.model_name: Optional[str] = None
    self.llm: Optional[LLM] = None
    self.tokenizer: Optional[AutoTokenizer] = None
```

19.2 init_op 方法

为每个 OP 初始化运行时状态，包括采样参数配置、模型切换（如果需要）、系统提示词设置。

19.2.1 模型切换逻辑

- 比较当前模型名称和新 OP 的模型名称
- 如果不同，释放旧模型并加载新模型
- 清空 CUDA 缓存以释放显存

19.3 execute 方法

执行一个 OP 的批量推理任务，这是 Worker 的核心方法。

19.3.1 执行流程

1. **初始化阶段**：调用 `init_op` 初始化 OP 状态
2. **输入构建**：根据是否使用聊天模板，构建输入
3. **批量推理**：调用 `llm.generate(inputs, self.sampling_params)`
4. **结果处理**：提取每个查询的生成文本，记录性能基准

19.4 run 方法

Worker 的主循环，持续监听命令队列并执行任务。

Part VII

Util 模块

20 Util 模块概述

util.py 模块提供了一些通用的工具函数，主要用于：

- 数据类型转换（字符串到 torch.dtype）
- GPU 设备管理（解析可见 GPU ID）

21 函数详解

21.1 __resolve__dtype

将字符串类型的数据类型规范转换为 torch.dtype 对象。

21.2 __visible__physical__gpu__ids

解析当前可见的物理 GPU ID 列表，考虑 CUDA_VISIBLE_DEVICES 环境变量的影响。

Part VIII

使用指南

22 基本使用

```python
from halo.optimizers import Optimizer
from halo.components import Query

# 创建优化器
opt = Optimizer("templates/adv_reason_3.yaml")

# 创建查询
queries = [
    Query(0, "What is Machine Learning?"),
    Query(1, "Explain neural networks."),
```

```
]

# 执行
opt.schedule(queries, strategy="heuristic")
queries = opt.execute(return_queries=True)

# 查看结果
for q in queries:
    print(f"Query {q.id} final output: {q.op_output}")

# 性能分析
opt.print_latency_percentiles()

# 清理
opt.exit()
```

23 性能优化建议

23.1 调度策略选择

- **简单工作流**：使用 `rr` 或 `by_levels`
- **多模型切换**：使用 `search` 或 `heuristic`
- **大批量查询**：使用 `dp`（数据并行）
- **缓存优化**：使用 `colocate_parents`

24 总结

Halo 系统是一个完整的智能体工作流执行框架，具有以下特点：

- **声明式配置**：通过 YAML 文件定义工作流，易于修改和扩展
- **多设备并行**：支持多 GPU 并行执行，提高吞吐量
- **智能调度**：多种调度策略，优化执行效率
- **依赖保证**：自动检查依赖关系，确保正确执行
- **高性能**：基于 vLLM，支持高效的批量推理